

Reviewer Pack — AC^0 PARITY Lower Bound via Linear Independence Invariant

Entry 0423a578c1a6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 05:07:19 UTC

AC^0 PARITY Lower Bound via Linear Independence Invariant

Entry ID: 0423a578c1a6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 05:07:19 UTC

1. Conjecture

Field A (mathematical branch): communication complexity **Field B** (complexity object): real algebraic geometry

Statement:

For any AC^0 circuit C computing PARITY on n inputs, the minimal number of linearly independent functions it can compute is $\Omega(\log \text{size}(C))$

Rationale (proposer’s reasoning):

This invariant leverages the linearity of PARITY and the structural limitations of AC^0 circuits. By measuring the circuit’s ability to represent linearly independent functions (a natural group action under variable permutations), we isolate the complexity inherent in computing PARITY. The logarithmic lower bound on this invariant would directly contradict the existence of small AC^0 circuits for PARITY.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0fde87952fb48822

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - AC⁰ PARITY lower bound linear independence real algebraic geometry - communication complexity real algebraic geometry linear independence invariant - AC⁰ circuits PARITY function polynomial equations real varieties

Top relevant hits considered: - [http://arxiv.org/abs/1409.1534v1] Algorithms in Real Algebraic Geometry: A Survey - [http://arxiv.org/abs/2506.14060v16] Linear Geometry and Algebra - [http://arxiv.org/abs/0912.2255v3] Test ideals via algebras of p^{-e} -linear maps - [http://arxiv.org/abs/1101.3737v2] Continuous rational functions on real and p -adic varieties - [http://arxiv.org/abs/1606.00596v3] Randomized Polynomial Time Identity Testing for Noncommutative Circuits - [http://arxiv.org/abs/1307.2463v2] Some equations for the universal Kummer variety

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_circuit(n, size):
        # Generate a random AC0 circuit with n inputs and given size
        circuit = []
        for _ in range(size):
            gate_type = random.choice(['AND', 'OR'])
            if gate_type == 'AND':
                inputs = [random.randint(0, 1) for _ in range(n)]
                circuit.append((gate_type, inputs))
            else:
                inputs = [random.randint(0, 1) for _ in range(n)]
                circuit.append((gate_type, inputs))
        return circuit

    def compute_linearly_independent_functions(circuit):
        # Compute the linearly independent functions using Fourier analysis
        n = len(circuit[0][1])
        count = 0
        for i in range(2**n):
            values = [circuit[j][1] for j in range(len(circuit)) if all(x == y for x, y in zip(bin(i), bin(y)))]
            if len(values) > count:
                count = len(values)
        return count

    def is_linearly_independent(f1, f2):
```

```

# Check if two functions are linearly independent
n = len(f1[1])
matrix = []
for i in range(2**n):
    row = [f1[j] for j in range(len(f1)) if all(x == y for x, y in zip(bin(i)[2:].zfill(n),
    row += [f2[j] for j in range(len(f2)) if all(x == y for x, y in zip(bin(i)[2:].zfill(n),
    matrix.append(row)
return gaussian_elimination(matrix) == 0

def gaussian_elimination(matrix):
    # Perform Gaussian elimination to check linear independence
    n = len(matrix[0]) // 2
    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(matrix[r][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        for j in range(i + 1, n):
            factor = matrix[j][i] / matrix[i][i]
            for k in range(2 * n):
                matrix[j][k] -= factor * matrix[i][k]
    return sum(abs(matrix[i][i]) for i in range(n))

def parity_function(x):
    # Compute the PARITY function
    return sum(x) % 2

n = random.randint(5, 40)
size = random.randint(1, 10 * n)
circuit = generate_ac0_circuit(n, size)
linearly_independent_count = compute_linearly_independent_functions(circuit)

conjecture_holds = linearly_independent_count >= math.log2(size)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "linearly_independent_count",
    "metric_value": linearly_independent_count,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:

```

```

    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run test with extended computation time

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	113115
2	propose	ollama_remote	qwen3:8b	0	0	120734
3	preregistration	ollama_remote	qwen3:8b	0	0	26049
4	novelty	ollama_remote	qwen3:8b	0	0	21218
5	novelty	ollama_remote	qwen3:8b	0	0	26049
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14711
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13828
8	judge	ollama_remote	qwen3:8b	0	0	18711

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 354416 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/0423a578c1a6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0423a578c1a6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0423a578c1a6.tar.gz` (if generated)